

Using Random Forest and Multi-Layer Perceptron for Adult Income Classification

Jinjia Kang (50091025)

Aberdeen Institute of Data Science and Artificial Intelligence
University of Aberdeen / SCNU

50091025

Yongdong Li (50091023)

Aberdeen Institute of Data Science and Artificial Intelligence
University of Aberdeen / SCNU

50091023

Abstract

This paper studies adult income prediction as a binary classification problem on the public Adult dataset. The task is to predict whether an individual's annual income exceeds \$50K from 14 demographic and employment-related attributes. We compare two substantially different machine learning methods covered in the module: Random Forest and Multi-Layer Perceptron (MLP). Both models are trained and evaluated under the same preprocessing pipeline, stratified 70/15/15 data split, and validation-based model selection protocol. On the held-out test set, Random Forest achieved higher Accuracy (0.8622), Precision (0.7812), and ROC-AUC (0.9141), while MLP reached a Precision of 0.7346 together with higher Recall (0.6269) and F1-score (0.6765). In practice, MLP finds more high-income cases, while Random Forest makes fewer false positive predictions. MLP is also much lighter, training faster (2.89s vs. 9.59s) and requiring far less memory and storage. For this task, Random Forest is a better fit when conservative positive predictions and simpler interpretation matter most, whereas MLP is more appealing when recall, compactness, and low computational cost matter more.

1. Introduction

Income prediction is a practical classification problem with applications in labour economics, policy analysis, and benefit targeting. It is also a useful machine learning benchmark because errors do not all have the same implication: a model that predicts too many people as high-income behaves very differently from one that misses too many true high-income cases. That makes the task suitable for comparing not only

overall accuracy, but also the type of mistakes made by different models.

In this project, we study the Adult income dataset from the UCI Machine Learning Repository [3, 4]. Our objective is to determine whether a Random Forest classifier [1] or a Multi-Layer Perceptron (MLP) [2, 5] is better suited to predicting whether an individual's income exceeds \$50K. This is a useful comparison because the dataset is structured and mixed-type, which usually favours tree-based methods, while still being large enough to test whether a neural network can remain competitive after careful preprocessing.

The main contributions of this paper are threefold. First, it builds a reproducible Adult income classification pipeline from preprocessing to final evaluation. Second, it compares two clearly different model families, a tree ensemble and a neural network, under the same split and model-selection rule. Third, it studies both predictive quality and practical cost, so the final recommendation is based on the type of error made as well as the resources required.

2. Problem Setting

We address a supervised binary classification problem. Let $\mathbf{x}$ denote a structured feature vector and let $y \in \{0, 1\}$ denote the income label, where 1 corresponds to annual income greater than \$50K. The goal is to learn a classifier $f(\mathbf{x})$ that predicts the income class for unseen examples.

The Adult dataset contains 48,842 rows with 14 predictive features and one binary income label [4]. Among the 14 features, 6 are numeric and 8 are categorical. The class distribution is moderately imbalanced: 11,687 positive examples ($>50K$) and 37,155 negative examples ($\leq 50K$). The features include age, work class, education, occupation, relationship, sex, capital gain, capital loss, and hours

worked per week. This dataset is appropriate for the selected task because its features describe real demographic and employment-related factors that can plausibly support income classification, while its mixed data types and class imbalance make the comparison more informative than a very clean toy dataset.

The preprocessing strategy follows the assessment brief. Missing values represented by “?” are treated as unavailable observations. Numeric features are imputed with the median, categorical features are imputed with the most frequent category, and categorical variables are transformed through one-hot encoding. Standardization is applied only for the MLP because neural networks are more sensitive to feature scale, whereas tree-based models are generally insensitive to monotonic feature scaling. No target information from the validation or test split is used during preprocessing, which helps avoid data leakage.

3. Methodology

Random Forest. Random Forest is an ensemble method that averages the outputs of multiple decision trees trained on bootstrapped data subsets [1]. For binary classification, its predicted positive-class probability can be written as $\hat{p}_{RF}(y = 1|\mathbf{x}) = \frac{1}{T} \sum_{t=1}^T h_t(\mathbf{x})$, where each tree h_t is trained on a resampled version of the training data. Random feature selection at splits reduces correlation between trees, while averaging lowers variance compared with a single decision tree. This is a sensible baseline for Adult because the data contain mixed numeric and categorical attributes, missing values handled by imputation, and likely non-linear threshold effects around variables such as age, capital gain, and working hours. In this project, the search space includes $n_estimators \in \{200, 500\}$, $max_depth \in \{\text{None}, 20, 40\}$, and $min_samples_leaf \in \{1, 5\}$. The best validation configuration was 500 trees with maximum depth 20 and minimum leaf size 1.

Multi-Layer Perceptron. The second method is a fully connected neural network trained with back-propagation [2]. After preprocessing, the MLP maps the input through hidden layers of the form $\mathbf{z}^{(l)} = \sigma(W^{(l)}\mathbf{z}^{(l-1)} + \mathbf{b}^{(l)})$ and outputs a probability for the positive class. Its parameters are learned by minimising cross-entropy loss with L2 regularisation, controlled by α . This allows the model to learn flexible non-linear decision boundaries after categorical expansion and feature scaling, which makes it a useful contrast to tree ensembles even though tabular problems are not always where neural networks work best. The explored configurations use hidden layer sizes (128, 64) and (256, 128), with $\alpha \in \{10^{-4}, 10^{-3}\}$, learning rate $\in \{10^{-3}, 5 \times 10^{-4}\}$, early stopping enabled, and a maximum of 100 training epochs. The

Table 1. Test-set classification metrics for the two compared models.

Model	Acc.	Prec.	Rec.	F1	AUC
Random Forest	0.8622	0.7812	0.5887	0.6714	0.9141
MLP	0.8566	0.7346	0.6269	0.6765	0.9087

best validation configuration used hidden layers (128, 64), $\alpha = 0.001$, and learning rate 0.001.

Why these two methods. The assessment requires two methods with substantially different modelling assumptions. Random Forest and MLP satisfy this requirement because one relies on many local decision rules aggregated across trees, while the other learns a shared representation through gradient-based optimisation. This gives a more informative comparison than choosing two closely related variants from the same model family.

4. Experimental Setup

All experiments use the same stratified train/validation/test split with proportions 70%/15%/15% and random seed 42, resulting in 34,189 training examples, 7,326 validation examples, and 7,327 test examples. Model selection is based on validation F1-score, while the test set is used only for the final comparison.

The implementation uses Python 3.12.7, scikit-learn, and a CPU-only Windows 11 environment with 31.82 GB RAM. The final reported metrics are Accuracy, Precision, Recall, F1-score, and ROC-AUC. To keep the comparison fair and reproducible, both methods use the same data split, the same cleaned inputs, the same random seed, and the same validation-driven model-selection rule. We do not apply class re-weighting or special threshold tuning to only one method, because that would blur the comparison. Early stopping is enabled for the MLP to reduce overfitting, while the Random Forest is controlled through the depth and leaf-size search grid.

In addition to predictive performance, we report training time, inference time per sample, memory growth during training, and serialized model size. This matters because two models with similar scores can still differ a lot in how costly they are to run.

5. Quantitative Evaluation

Table 1 reports the final test-set metrics. Random Forest achieved the strongest Accuracy, Precision, and ROC-AUC, while MLP achieved stronger Recall and a slightly better F1-score.

Table 2 shows the error pattern behind the precision-recall trade-off. MLP recovers more positive cases, while

Table 2. Confusion matrix counts on the held-out test set.

Model	TN	FP	FN	TP
Random Forest	5285	289	721	1032
MLP	5177	397	654	1099

Table 3. Complexity and efficiency comparison on the held-out test set.

Model	Train (s)	Infer (ms)	Mem. (MB)	Size (MB)
Random Forest	9.5859	0.0446	175.7930	170.7053
MLP	2.8893	0.0019	0.9766	0.5150

Random Forest avoids more false alarms.

Table 3 summarises the computational trade-offs. MLP is substantially more efficient: it trains in about one third of the Random Forest time, uses far less memory, performs much faster inference, and produces a dramatically smaller model. This difference is expected because the Random Forest must store and query 500 decision trees, whereas the MLP stores a compact set of dense weight matrices.

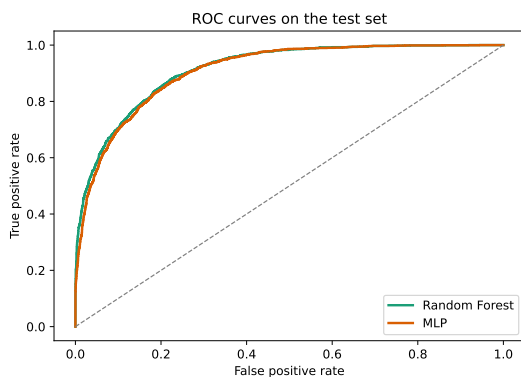


Figure 1. ROC curves for the two models on the test split.

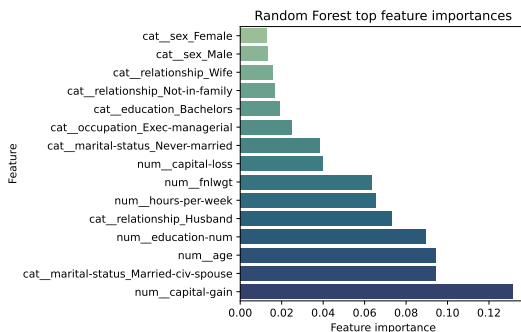


Figure 2. Random Forest feature importance.

Discussion. The comparison shows a trade-off rather than a simple winner. Relative to MLP, Random Forest improves Precision by 4.66 percentage points and ROC-AUC by 0.54 points, but loses 3.82 points of Recall and 0.51 points of F1. The confusion-matrix pattern makes this especially clear: the MLP identifies 67 additional true high-income cases on the test set, but it also introduces 108 more false positives. In other words, MLP is more willing to predict the positive class, while Random Forest is more conservative.

The ROC curves in Figure 1 show that both models separate the classes well overall, with only a modest gap between them. The bigger difference comes from how they behave at the default decision threshold. Random Forest is more cautious: when it predicts income above \$50K, it is more often correct, but it also leaves more positive cases undiscovered. The MLP moves the other way, giving up some precision to recover more of the minority class.

Figure 2 makes the Random Forest result easier to interpret. The most influential features are capital gain, marital-status indicators, age, education-num, relationship, and hours-per-week. These variables make sense for the Adult task and fit the intuition that income is related to accumulated earnings signals, life stage, and labour-market position. At the same time, the importance ranking should not be read as causal evidence; it only shows which variables helped this model split the dataset most effectively.

MLP is therefore the better default choice when storage, memory use, and recall matter most, especially because it has the best F1-score and a very small computational footprint. Random Forest is still preferable when conservative positive predictions, slightly stronger discrimination, and easier explanation matter more.

6. Conclusion

This project compared Random Forest and MLP on the Adult income prediction task using a shared preprocessing and evaluation pipeline. The main finding is that model choice changes the type of errors made, not just the headline score. Random Forest achieved higher Accuracy, Precision, and ROC-AUC, while MLP achieved higher Recall and F1-score. In addition, MLP was much faster to train and infer with, used far less memory, and produced a much smaller model file.

For this dataset, MLP is the better overall recommendation if the goal is a compact and efficient classifier that still captures a reasonable share of the positive class. Random Forest is the stronger option when conservative positive predictions and easier model inspection matter more than footprint. In short, the better model depends on whether the application is more sensitive to false positives or missed positives.

The main limitations of this study are the restricted hyperparameter search space and the fact that only two model

families were compared. In addition, the analysis stays at the level of aggregate predictive performance, so it does not address fairness, calibration, or subgroup-specific behaviour. Useful next steps would include tuning decision thresholds for different precision-recall targets, adding a gradient-boosted tree baseline because it is often strong on tabular data, and checking error rates across demographic groups to see whether the aggregate results hide uneven behaviour.

References

- [1] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. [1](#), [2](#)
- [2] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. [1](#), [2](#)
- [3] Ron Kohavi. Scaling up the accuracy of naive-bayes classifiers: a decision-tree hybrid. In *KDD*, pages 202–207, 1996. [1](#)
- [4] UCI Machine Learning Repository. Adult data set, 1996. <https://archive.ics.uci.edu/dataset/2/adult>. [1](#)
- [5] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[1](#)